DATA LAKE ARCHITECTURE

A Comprehensive Design Proposal

Medical Data Processing Company

Document Information	
Document Title	Data Lake Architecture — A Comprehensive Design Document
Company	Medical Data Processing Company, San Francisco, CA
Author	Data Architect — Edward Amankwah
Version	1.0
Date	2024
Classification	CONFIDENTIAL
Status	Final for Executive Review

1. Purpose

This document presents the comprehensive Data Lake architecture design for Medical Data Processing Company, a San Francisco-based provider of Electronic Medical Record (EMR) analytics serving over 8,000 medical facilities across the United States. The purpose is to propose an open-source, scalable, and fault-tolerant Data Lake solution that addresses the company's critical infrastructure failures and enables future growth.

This document is prepared in response to the directive from the Chief Technology Officer (CTO) to evaluate whether a Data Lake implementation can resolve the company's current scalability, reliability, and analytical capability gaps. It provides design rationale, architectural decisions, implementation considerations, risk analysis, and supporting evidence for the proposed solution.

Target Audience

This document targets the technical audience of Medical Data Processing Company, including the CTO, Senior Engineers, Data Engineers, Database Administrators, and Solution Architects who will be involved in reviewing, approving, and implementing the proposed solution.

In-Scope Items

- Design of a 4-layer Data Lake architecture: Ingestion, Storage, Processing, and Serving
- Open-source technology stack selection and justification
- Data ingestion strategy for FTP, REST APIs, streaming, and legacy SQL Server
- Storage zone design (Raw, Staging, Curated, Archive) using HDFS
- Batch and stream processing architecture using Apache Spark and Kafka Streams
- Metadata management, data catalog, and lineage tracking using Apache Atlas
- Security, governance, and HIPAA compliance design
- High availability, disaster recovery, and backup strategies
- Serving layer design for dashboards, reports, ML, and APIs
- Hot vs. cold data archival strategy
- Alternative tool evaluation for each layer

Out-of-Scope Items

- Application-level development (customer portal UI rebuild)
- Cloud migration strategy (solution is designed for on-premise deployment)
- Decommissioning plan for legacy SQL Server systems
- HIPAA legal compliance audit (regulatory legal review is out of scope)
- Third-party vendor licensing negotiations
- Network infrastructure upgrades below the application layer

2. Requirements

2.1 Problem Statement Summary

Medical Data Processing Company processes Electronic Medical Records (EMR) for over 8,000 medical facilities. Since the company's founding in 2007, its infrastructure has been built on a traditional monolithic 3-tier architecture backed by Microsoft SQL Server — a proprietary, single-node database system. While the company has experienced hyper-growth over the past three years, infrastructure investment has not kept pace with data volume growth.

The system currently stores over 8TB of data in SQL Server (running at 70% storage capacity) and processes an average of 15 million data files per day arriving as compressed CSV, XML, and TXT files from FTP servers, REST APIs, and customer-hosted file servers. With a 15-20% year-over-year data growth rate, the existing infrastructure will be completely saturated within 2-3 years even with no additional growth.

Last week, a data surge during the nightly ETL process caused a complete database crash, taking the entire system offline for several hours — disrupting services to over 8,000 medical facilities. This event underscores the urgency of building a resilient, scalable platform.

2.2 Existing Technical Environment

Component	Specification
Master SQL DB Server	64-core vCPU, 512 GB RAM, 12 TB disk (70% full ~8.4 TB)
Stage SQL DB Server	Separate instance for staging data during ETL
ETL Jobs	70+ custom ETL jobs managing 100+ tables (SSIS packages)
Ingestion Servers	3 servers: FTP Server, Data Extract agents, API agents
Web/App Servers	32 GB RAM, 16-core vCPU each
Data Volume	8TB total; 15M files/day; 77K zip files/day
Data Growth	15-20% Year-over-Year
Data Sources	8,000 facilities: CSV, TXT, XML via FTP/API/Push

2.3 Business Requirements

- Improve overall system uptime and eliminate single points of failure
- Reduce latency of SQL queries and reporting pipelines
- Build a reliable, fault-tolerant, and self-healing architecture
- Scale horizontally as data volume and velocity increase (15-20% YoY)
- Improve business agility through automation and new framework support

- Embrace open-source tools exclusively — avoid proprietary/vendor lock-in solutions
- Metadata-driven design: common scripts to process all data source types
- Centrally store all enterprise data and enable easy cross-departmental access

2.4 Technical Requirements

- Process incoming files in near-real-time (replace nightly batch-only model)
- Separate metadata, data storage, and compute/processing layers
- Retain unlimited historical data without performance degradation
- Scale processing capacity horizontally with increasing data volume
- Sustain individual node failures without system downtime (fault tolerance)
- Support Change Data Capture (CDC) and UPSERT operations on key tables
- Drive multiple use cases from the same dataset without data movement
- Integrate with ML frameworks (TensorFlow, Spark MLlib)
- Support dashboarding tools (PowerBI, Tableau, Apache Superset)
- Enable ad-hoc SQL querying capability across the data lake

3. Data Lake Architecture Design Principles

Principle 1: Open Source First — Avoid Vendor Lock-in

All components in this architecture are selected from the open-source ecosystem (Apache Software Foundation ecosystem). This directly addresses the company's business requirement to avoid proprietary solutions. Open-source tools such as Apache Hadoop, Spark, Kafka, Hive, NiFi, and Atlas offer proven enterprise-grade reliability, active community support, and the flexibility to run on any hardware without vendor licensing fees. This reduces total cost of ownership and provides long-term architectural freedom.

Principle 2: Separation of Storage and Compute

The architecture strictly separates storage (HDFS) from compute (Apache Spark, YARN). This enables the company to scale storage and processing independently based on demand. When data volume grows, additional storage nodes can be added to the HDFS cluster without needing to provision additional compute resources, and vice versa. This is critical given the 15-20% annual data growth rate. It also allows different processing engines (Hive, Impala, Presto, Spark) to operate against the same data simultaneously.

Principle 3: Metadata-Driven Design

Rather than building custom ETL scripts for each of the 100+ data source types (as exists today with 70+ SSIS packages), the new architecture adopts a metadata-driven approach. A centralized metadata store (Apache Atlas + Hive Metastore) defines schema, transformation rules, and processing configurations. A common set of PySpark jobs reads this metadata and applies the appropriate processing logic dynamically. This dramatically reduces maintenance overhead, accelerates onboarding of new data sources, and enables self-service data discovery.

Principle 4: Fault Tolerance and High Availability

Every layer of the architecture is designed with redundancy. HDFS maintains a replication factor of 3, ensuring no data loss on node failure. Apache Kafka uses partition replication for stream ingestion resilience. ZooKeeper manages leader election for HDFS NameNode High Availability. YARN manages distributed resource allocation. This eliminates the current single-point-of-failure risk posed by the monolithic SQL Server.

Principle 5: Schema-on-Read and Data Immutability

The Raw Zone of the data lake stores data in its original, unmodified form (schema-on-read). Data is never deleted from the raw zone, providing a permanent audit trail critical for HIPAA compliance. Transformations and schema enforcement happen at the Staging and Curated zones. This architecture supports time-travel queries via Apache Hudi, enabling the company to query historical states of data — a requirement for compliance reporting.

4. Assumptions and Risks

4.1 Assumptions

#	Assumption	Design Impact	Risk if Incorrect
1	The company has on-premise data center infrastructure (servers/racks) capable of hosting a Hadoop cluster of at least 10 nodes	Architecture is designed as on-premise HDFS deployment — no cloud vendor components	If no data center exists, cloud migration planning would be required, significantly changing architecture
2	Network bandwidth between the 8,000 medical facilities and the company's data center is sufficient (minimum 1 Gbps internal network) to support real-time data ingestion via Kafka	Kafka-based streaming ingestion is designed assuming network capacity for 700K files/hour	Insufficient bandwidth would require buffered batch ingestion, reducing real-time capability
3	A 3-replication factor in HDFS provides adequate data durability; the company does not require geo-redundant replication across data centers	Single data center HDFS cluster with local replication is designed	If geo-redundancy is required, DistCp-based cross-cluster replication must be added
4	Existing ETL business logic (transformation rules) is documented or accessible and can be re-implemented as PySpark jobs	The metadata-driven PySpark framework assumes transformations can be parameterized	If business logic is buried in undocumented SSIS packages, significant reverse-engineering effort is needed
5	Data from facilities is pseudonymized or de-identified at the source per HIPAA Safe Harbor rules; the data lake stores only permissible data	Security design includes RBAC and encryption but does not include a PHI de-identification pipeline	If raw PHI data is ingested, additional HIPAA-compliant de-identification tooling is required

6	The company has or can hire data engineers proficient in PySpark, Hadoop, and Kafka	Architecture assumes in-house operational capability for open-source stack management	Talent gaps would require external consulting during implementation phase

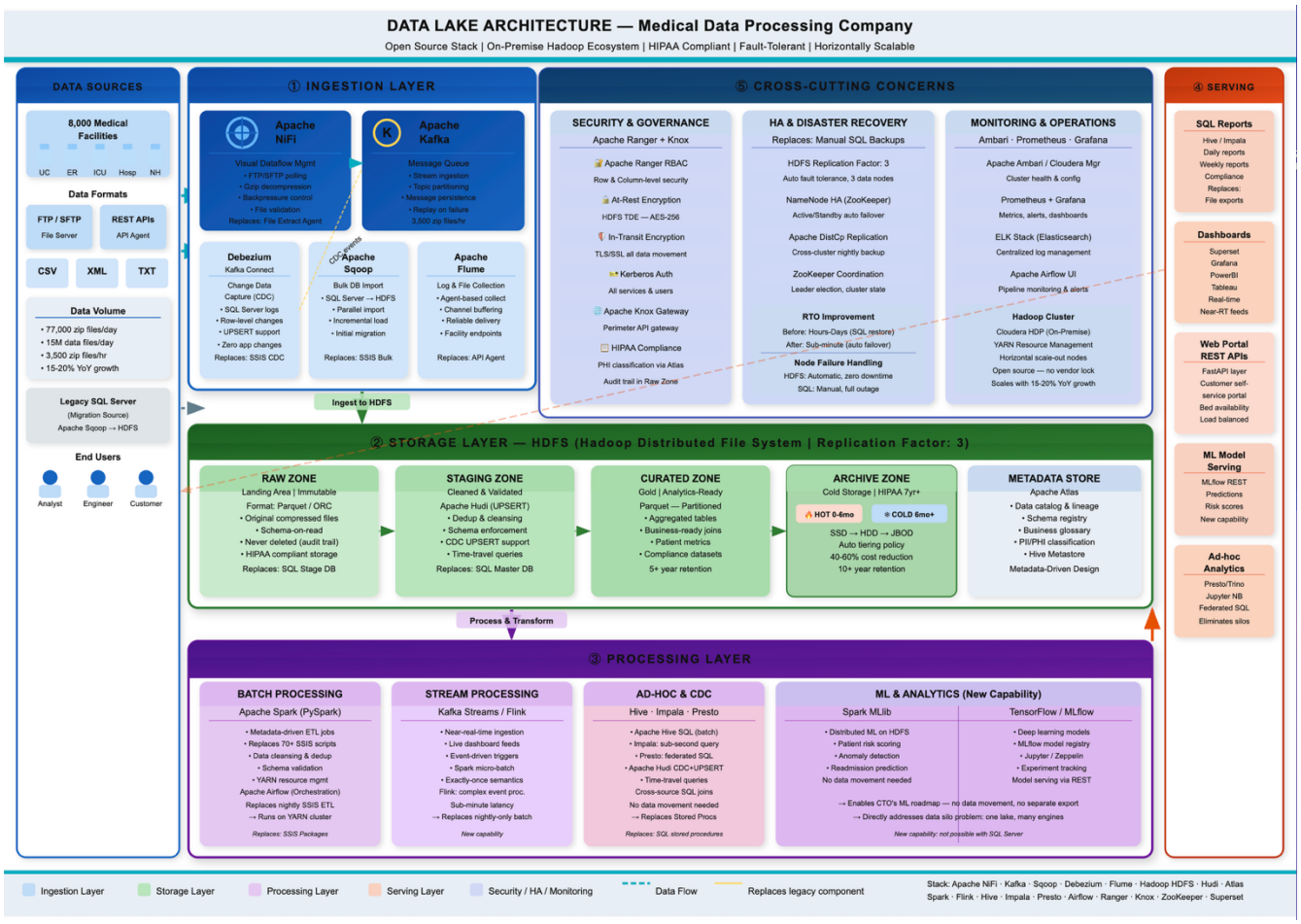
4.2 Potential Risks

- Migration Risk: Migrating 8TB of data from SQL Server to HDFS while keeping systems live requires careful cutover planning. A parallel-run strategy (running both systems simultaneously during migration) mitigates this risk.
- Operational Complexity: Managing an open-source Hadoop ecosystem requires specialized expertise. Tools like Cloudera Manager or Apache Ambari simplify operations but add administrative overhead compared to managed cloud services.
- Data Quality Risk: Without schema enforcement at ingestion, poor-quality or malformed data entering the Raw Zone can propagate issues downstream. Apache NiFi validation rules and Great Expectations (data quality framework) should be implemented.
- HIPAA Compliance Risk: Any misconfiguration of Apache Ranger policies could expose protected health information. Regular security audits and penetration testing are recommended.
- Hadoop NameNode Risk: Despite HA configuration, NameNode is still a complex component. ZooKeeper failover must be tested regularly through chaos engineering exercises.

5. Data Lake Architecture for Medical Data Processing Company

The proposed architecture follows a four-layer medallion pattern: Ingestion Layer, Storage Layer (Raw → Staging → Curated → Archive), Processing Layer, and Serving Layer. All components are open-source and designed for on-premise deployment using the Apache Software Foundation ecosystem. The architecture is horizontally scalable, fault-tolerant, and HIPAA-compliant.

See the accompanying architecture diagram below for the complete visual representation of all components, data flows, and layer boundaries. The diagram shows all four layers with tool logos/names, metadata storage location (Apache Atlas), and the cross-cutting security/monitoring concerns.



6. Design Considerations and Rationale

6.1 Ingestion Layer

Overview

The ingestion layer is responsible for collecting data from 8,000+ medical facilities arriving in multiple formats (CSV, TXT, XML) via FTP servers, REST APIs, and direct pushes. It must handle 77,000 zip files per day (averaging 3,500 per hour) and scale to support 15-20% annual growth.

Primary Tool: Apache NiFi

Apache NiFi is selected as the primary data ingestion orchestration tool. NiFi provides a visual dataflow management system with built-in processors for FTP/SFTP polling, REST API calls, file decompression, and routing. NiFi's backpressure mechanism prevents the system from being overwhelmed during data surges — directly addressing the crash scenario that took down SQL Server. NiFi supports metadata-driven flow configuration, enabling a single generic flow template to handle multiple data source types without custom scripting.

- NiFi handles: SFTP/FTP polling, gzip decompression, file validation, routing to Kafka or HDFS
- NiFi Registry provides version control for flow configurations
- Horizontal scaling: NiFi cluster can add nodes to handle increased throughput

Stream Ingestion: Apache Kafka

Apache Kafka serves as the high-throughput, distributed message queue between NiFi and the storage/processing layers. Each data file type (CSV, XML, TXT, per-facility) maps to a Kafka topic. Kafka's partitioning model allows parallel processing by multiple Spark Streaming consumers simultaneously. Kafka Streams enables real-time processing for near-real-time dashboard requirements. Kafka provides message persistence, enabling replay of failed messages without data loss.

Database Migration: Apache Sqoop

Apache Sqoop is used for bulk import of historical data from the existing SQL Server database during the initial migration phase. Sqoop supports parallel import jobs and can be configured to perform incremental imports, capturing only changed records after the initial full load.

Change Data Capture: Debezium + Kafka Connect

Debezium (via Kafka Connect) captures row-level changes from the legacy SQL Server database in real-time using transaction log reading. This enables CDC without modifying the source application, providing UPSERT capability required in the technical requirements.

File Collection: Apache Flume

Apache Flume is used for collecting log files and streaming file-based data from agent-based data collectors deployed at facility endpoints. Flume's channel-based architecture provides reliable delivery with channel buffering.

Tool	Status	Reason Not Selected

Logstash (ELK)	Considered	Better suited for log analytics; lacks native FTP/SFTP processing and flow management UI; no backpressure support
StreamSets	Considered	Proprietary enterprise licensing; violates open-source-first principle despite having an open-source version
AWS Kinesis	Rejected	Cloud-only; violates vendor lock-in requirement; not available for on-premise deployment
Azure Data Factory	Rejected	Cloud-only proprietary service; directly violates the open-source mandate
Apache Storm	Considered	Lower-level than Kafka Streams/Flink; requires more boilerplate code; less mature ecosystem for this use case

6.2 Storage Layer

Primary Platform: HDFS (Hadoop Distributed File System)

HDFS is selected as the foundational storage platform. HDFS is designed for petabyte-scale storage with a replication factor of 3 (three copies of every data block across different nodes), providing automatic fault tolerance. Unlike SQL Server's single-node architecture, HDFS distributes data across a cluster of commodity servers, eliminating the single-point-of-failure risk. HDFS supports unlimited horizontal scaling — adding storage nodes increases capacity without service interruption.

Storage Zone Design (Medallion Architecture)

Data progresses through four storage zones, each serving a distinct purpose:

Zone	Purpose	Data State	Technology	Retention
Raw Zone	Immutable landing area	Original compressed files, schema-on-read	HDFS + Parquet/ORC	Unlimited (permanent)
Staging Zone	Cleaned and validated data	Deduplicated, schema-enforced, UPSERT-capable	HDFS + Apache Hudi	Rolling 2-year window
Curated Zone	Business-ready analytics data	Aggregated, joined, enriched	HDFS + Parquet (partitioned)	5+ years
Archive Zone	Cold data long-term retention	Compressed historical data	HDFS (cold tier / JBOD)	10+ years

Hot vs. Cold Data Strategy

Hot data (last 6 months) is stored on high-performance SSD-backed HDFS data nodes, optimized for fast query response. Cold data (older than 6 months) is automatically tiered to high-capacity, lower-cost JBOD (Just a Bunch of Disks) HDFS nodes using HDFS storage policies. Apache Ozone can be used for object-store-compatible cold storage. This tiering policy reduces infrastructure costs while maintaining full data accessibility.

Data Format: Apache Parquet

Apache Parquet is selected as the primary columnar storage format. Parquet provides 60-80% storage compression compared to raw CSV/XML, critical for managing the 15-20% annual data growth. Columnar storage enables analytical queries to read only the required columns, dramatically reducing I/O and improving query performance. Parquet is natively supported by Spark, Hive, Impala, and Presto. ORC (Optimized Row Columnar) format is used in Hive-specific workloads for additional ACID transaction support.

UPSERT and CDC Support: Apache Hudi

Apache Hudi (Hadoop Upserts Deletes and Incrementals) is implemented in the Staging Zone to support the CDC and UPSERT technical requirements. Hudi maintains a transaction log and supports time-travel queries, enabling the company to query historical states of patient records for compliance reporting.

Metadata Management: Apache Atlas

Apache Atlas provides enterprise-grade metadata management and data governance. Atlas automatically captures data lineage (how data flows from source to serving layer), maintains a business glossary, and integrates with Apache Ranger for policy-based access control. Every dataset in HDFS is registered in Atlas with schema information, owner, classification (PII, PHI), and lineage graph.

Security

- At-Rest Encryption: HDFS Transparent Data Encryption (TDE) using AES-256 encryption for all zones containing PHI
- In-Transit Encryption: TLS/SSL for all data movement between NiFi, Kafka, HDFS, and Spark
- Authentication: Kerberos for cluster-level authentication of all services and users
- Authorization: Apache Ranger for fine-grained Row and Column-level security, RBAC policies aligned with HIPAA minimum-necessary principle
- Network Security: Apache Knox as perimeter gateway for external API access to the cluster

Backup and Recovery

- HDFS Replication Factor 3: Automatic local fault tolerance across 3 data nodes
- Cross-Cluster Replication: Apache DistCp (Distributed Copy) replicates critical datasets to a secondary HDFS cluster in a separate physical location nightly
- NameNode High Availability: ZooKeeper-based automatic failover between active and standby NameNode eliminates NameNode as SPOF
- Recovery Time Objective (RTO): Sub-minute failover for node failures; 1-2 hours for full cluster recovery vs. hours/days in current SQL Server model

Tool	Status	Reason Not Selected
Amazon S3	Rejected	Cloud-only; violates vendor lock-in requirement
MinIO	Considered	Viable S3-compatible on-prem object store; limited CDC/UPSERT support without additional tooling; HDFS chosen for tighter Hadoop ecosystem integration
Apache HBase	Considered	Excellent for random read/write on key-value data; not optimal for large-scale analytical queries; would add architectural complexity
Cassandra	Considered	Excellent write performance; lacks SQL support and analytics integration; does not support complex analytical queries natively
Delta Lake	Considered	Excellent ACID transactions and time-travel; requires Spark runtime; Hudi chosen for broader compatibility with Hive, Impala, and Presto without Spark dependency

6.3 Processing Layer

Primary Engine: Apache Spark

Apache Spark is selected as the unified batch and stream processing engine. Spark runs on YARN (Yet Another Resource Negotiator), the Hadoop resource manager, enabling shared cluster resource management with other workloads. Spark's in-memory processing delivers 10-100x performance improvements over traditional MapReduce. PySpark (Python API for Spark) is selected as the development language, given its broad adoption in the data engineering community and ML ecosystem compatibility.

Batch Processing

Spark batch jobs replace the 70+ SSIS packages with a metadata-driven PySpark framework. A single parameterized Spark job reads transformation metadata from Apache Atlas/Hive Metastore and applies the appropriate transformations (cleansing, deduplication, schema validation, aggregation) for each data source type. This reduces maintenance burden and accelerates onboarding of new facility data formats. Apache Airflow orchestrates Spark job scheduling, dependencies, and retry logic.

Stream Processing: Kafka Streams and Apache Flink

Kafka Streams (embedded in the Kafka ecosystem) handles lightweight stateful stream processing for real-time alerting and near-real-time dashboards. Apache Flink is available for complex event processing scenarios requiring exactly-once semantics and event-time processing. Spark Structured Streaming provides micro-batch processing (sub-minute latency) for near-real-time use cases that don't require millisecond response times.

Ad-Hoc Querying

Multiple SQL engines are deployed to serve different query profiles:

- Apache Hive: Batch SQL for complex ETL-style queries, supports all HDFS data, excellent for large dataset scans
- Apache Impala: Low-latency interactive SQL on HDFS/Kudu, sub-second response for dashboard queries on the Curated Zone
- Presto/Trino: Federated query engine enabling SQL across HDFS, relational databases, and other sources simultaneously; ideal for ad-hoc analytics by data analysts

CDC Processing

Debezium CDC events captured in Kafka are consumed by Spark Structured Streaming, which applies UPSERT operations to Apache Hudi tables in the Staging Zone. This provides near-real-time synchronization of changes from the legacy SQL Server during the migration phase and ongoing CDC from any connected operational database.

ML and Analytics

Spark MLlib provides distributed machine learning algorithms running natively on the data lake without data movement. TensorFlow is integrated via TensorFlow on YARN for deep learning models. Apache Zeppelin and Jupyter Notebooks provide interactive data exploration environments for data scientists. MLflow manages model versioning, experiment tracking, and model registry.

Workflow Orchestration: Apache Airflow

Apache Airflow manages DAG-based workflow orchestration for all batch processing pipelines. Airflow provides dependency management, scheduling, retry logic, alerting on failures, and a web UI for pipeline monitoring. This replaces the fragile cron-based SSIS scheduling in the current environment.

Tool	Status	Reason Not Selected
Hadoop MapReduce	Considered	Disk-based; 10-100x slower than Spark; being deprecated in most modern Hadoop deployments; no streaming support
Apache Storm	Considered	Lower-level streaming framework; less expressive than Kafka Streams/Flink; lacks SQL abstraction; higher development overhead
Talend Open Studio	Considered	Open-source version has limited features; community edition lacks cluster deployment; not suitable for production-scale processing
Google Dataflow / Apache Beam	Considered	Beam is vendor-neutral runner; however, primary runners (Dataflow, Flink) add complexity; Spark chosen for ecosystem unification
Dask	Considered	Python-native distributed computing; smaller ecosystem than Spark; less mature for petabyte-scale workloads; better suited for data science notebooks

6.4 Serving Layer

What is the Serving Layer?

The Serving Layer is the final consumer-facing layer of the Data Lake. It exposes processed, curated data to end users, applications, dashboards, and APIs. Unlike the storage and processing layers (which deal with raw and intermediate data), the Serving Layer delivers ready-to-consume, business-aligned data products tailored to specific use cases.

Types of Data Stored

- Pre-aggregated summary tables (patient metrics, compliance statistics, bed availability counts) in the Curated HDFS zone
- Materialized views and query results cached in Apache Impala for low-latency dashboard access
- ML model scores and predictions stored as Parquet tables and served via REST APIs
- Compliance reports generated as structured data files (CSV/Excel) for customer download
- Real-time event streams from Kafka for near-real-time dashboard feeds

How Data is Used

- SQL Reports: Apache Hive and Impala serve daily, weekly, and nightly compliance and operational reports. Customers access standardized reports through the web portal.
- Dashboards: Apache Superset (open-source BI tool) serves as the primary dashboard platform, replacing proprietary BI tools. Tableau and PowerBI can connect via ODBC/JDBC to Impala or Presto. Grafana serves infrastructure and operational metrics dashboards.
- REST APIs: A FastAPI-based microservice layer queries Impala/Presto and serves results to the customer web portal and mobile applications. This decouples the presentation layer from the data layer.
- ML Model Serving: MLflow serves trained models via REST endpoints. Patient risk scores and predictive analytics are computed in batch (Spark) and served via API for real-time lookup.
- Ad-Hoc Analytics: Presto/Trino enables data analysts to run federated SQL queries across HDFS zones, legacy databases, and external sources through a unified SQL interface — eliminating data silos.

7. Standout: Alternative Tools, Archival Strategy, and Industry References

7.1 Alternative Tools Comparison by Layer

Layer	Selected Tool	Alternative 1	Alternative 2	Key Trade-off
Ingestion	Apache NiFi	Logstash (ELK)	Apache Flume	NiFi: Visual UI + backpressure; Logstash: log-centric; Flume: simpler but limited routing
Ingestion Stream	Apache Kafka	Apache Pulsar	RabbitMQ	Kafka: proven at scale; Pulsar: multi-tenant native; RabbitMQ: lower throughput, AMQP-focused
Storage	HDFS + Parquet	MinIO (S3-compat)	Apache Iceberg	HDFS: deep Hadoop integration; MinIO: cloud-native API; Iceberg: table format, not storage system
UPSERT/CDC	Apache Hudi	Delta Lake	Apache Iceberg	Hudi: Hadoop-native; Delta: Databricks-optimized; Iceberg: catalog-focused, broader engine support
Batch Processing	Apache Spark	Hadoop MapReduce	Dask (Python)	Spark: in-memory speed; MapReduce: disk-based, slow; Dask: Python-native but less mature
Stream Processing	Kafka Streams	Apache Flink	Apache Storm	Flink: true streaming, exactly-once; Kafka Streams: embedded, no cluster; Storm: older, low-level
Ad-hoc SQL	Presto/Trino	Apache Impala	Apache Hive	Presto: federated; Impala: fastest on HDFS; Hive: most compatible but slow interactive
Metadata	Apache Atlas	Apache Amundsen	DataHub (LinkedIn)	Atlas: deep Hadoop integration; Amundsen: search-first; DataHub: broader connector ecosystem
Orchestration	Apache Airflow	Apache Oozie	Prefect	Airflow: Python DAGs, rich UI; Oozie: XML-config, legacy;

				Prefect: modern but smaller community
Security	Apache Ranger	Apache Sentry	OpenLDAP	Ranger: fine-grained RBAC + UI; Sentry: Impala-focused; OpenLDAP: directory only, no data policies
BI/Dashboard	Apache Superset	Grafana	Redash	Superset: full BI; Grafana: metrics/time-series; Redash: simpler, SQL-first

7.2 Hot vs. Cold Data Archival Strategy

Given the 15-20% annual data growth, a tiered storage strategy is essential to control infrastructure costs while maintaining data accessibility.

Tier	Data Age	Storage Type	Access Frequency	Query Latency	Cost
Hot	0-6 months	SSD-backed HDFS nodes	Daily/Real-time	Seconds	High
Warm	6-24 months	HDD HDFS data nodes	Weekly/Monthly	Minutes	Medium
Cold	2+ years	JBOD/High-density HDD HDFS	Quarterly/Annual	Hours	Low
Archive	7+ years (HIPAA)	Tape/Remote HDFS cluster	Rare (compliance)	Days	Very Low

HDFS Storage Policies (HOT, WARM, COLD, ALL_SSD) are configured to automatically migrate data between storage tiers based on file age and last-access time. Apache Atlas policies define data classification (PHI, non-PHI) which determines minimum retention periods per HIPAA requirements (6 years for medical records). The tiered approach is expected to reduce storage costs by 40-60% compared to uniform SSD storage.

7.3 Industry References: Successful Data Lake Implementations

Company	Industry	Technologies Used	Key Outcome
Netflix	Media/Streaming	Apache Spark, Apache Iceberg, Hive, Presto	Processes petabytes of viewing data daily; enables personalization at scale for

			230M+ subscribers; eliminated data silos across 100+ microservices
Uber	Ride-sharing/Tech	Apache Kafka, Hadoop, Spark, Presto, Hudi	Apache Hudi was invented at Uber to solve CDC/UPSERT at scale; processes 1TB+ of data per hour; enabled real-time driver/rider matching analytics
LinkedIn	Professional Network	Apache Kafka, Hadoop, Samza, DataHub	Kafka was invented at LinkedIn; DataHub provides open-source metadata management now used by many enterprises; processes trillions of events per day
NYU Langone Health	Healthcare	Hadoop, Spark, Kafka, HIPAA-compliant lake	Built a HIPAA-compliant medical data lake for clinical analytics; enabled real-time patient monitoring dashboards reducing ICU mortality rates
Mayo Clinic	Healthcare/Research	Apache Hadoop, Spark, open-source ML stack	Integrated EMR data lake enabling genomic research and predictive diagnostics; processes billions of clinical data points for AI-assisted diagnostics

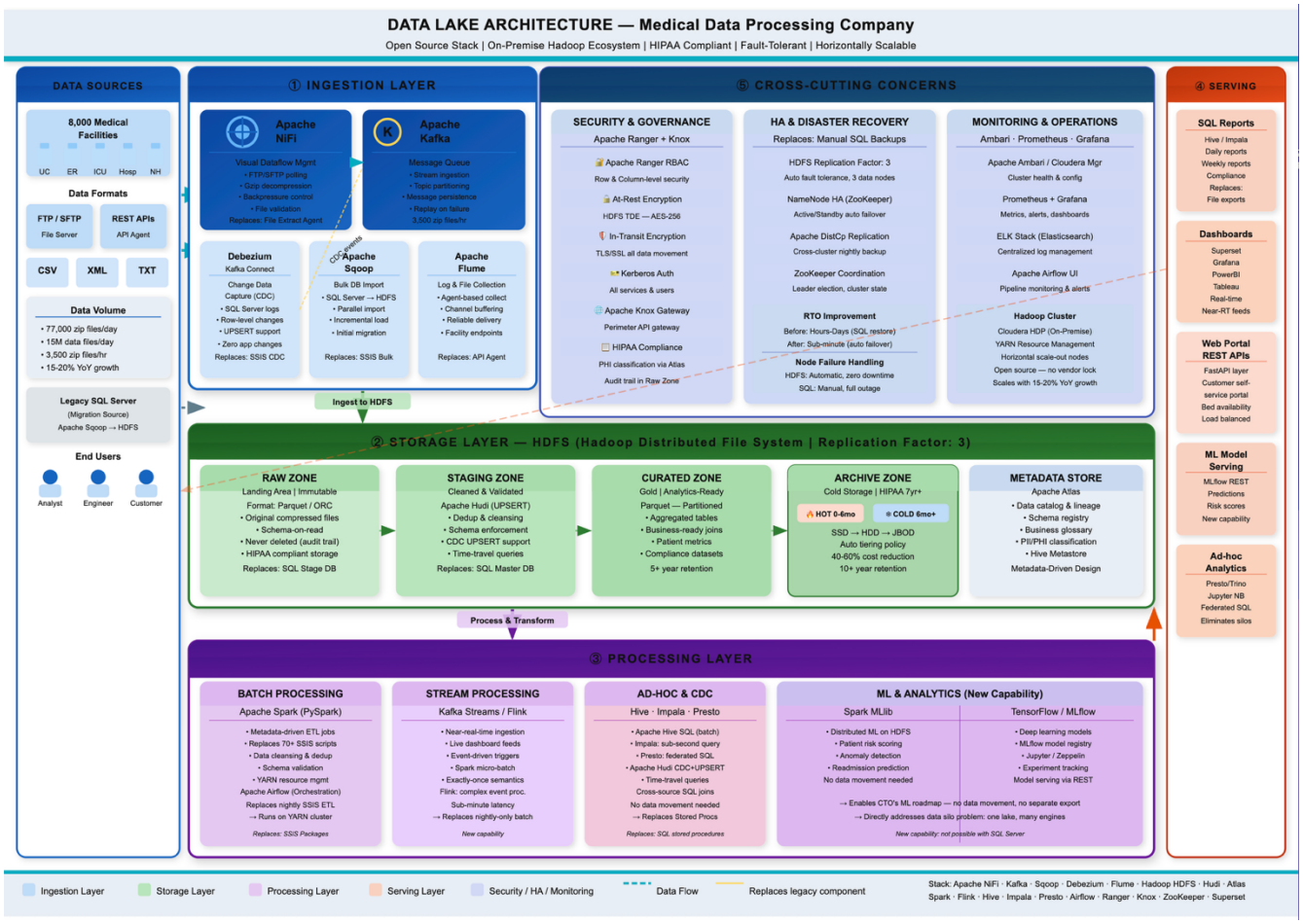
8. Conclusion

The proposed open-source Data Lake architecture provides Medical Data Processing Company with a scalable, fault-tolerant, and cost-effective path forward. By replacing the monolithic SQL Server with a horizontally scalable HDFS-based data lake powered by Apache Spark, Kafka, NiFi, and Hudi, the company eliminates its critical single points of failure, gains near-real-time processing capability, and enables unlimited data retention at manageable cost.

The metadata-driven design dramatically reduces operational complexity (from 70+ custom scripts to a unified PySpark framework), while Apache Atlas and Ranger provide the governance and security posture required for HIPAA compliance. The tiered hot/warm/cold storage strategy manages the 15-20% annual data growth sustainably. Recommended next steps include forming a project steering committee, conducting a detailed infrastructure capacity assessment, developing a phased migration plan starting with a parallel-run pilot on 10% of facilities, and engaging a Hadoop ecosystem implementation partner for production deployment.

9. References

- Apache Software Foundation — Hadoop Ecosystem: https://hadoop.apache.org
- Apache Kafka Documentation: https://kafka.apache.org/documentation
- Apache NiFi Documentation: https://nifi.apache.org/docs.html
- Apache Spark Documentation: https://spark.apache.org/docs/latest
- Apache Hudi — Uber Engineering Blog: https://hudi.apache.org
- Apache Atlas — Data Governance: https://atlas.apache.org
- Apache Ranger — Security: https://ranger.apache.org
- Debezium CDC: https://debezium.io/documentation
- Presto/Trino Documentation: https://trino.io/docs/current
- Apache Airflow: https://airflow.apache.org/docs
- Netflix Tech Blog — Data Lake Architecture: https://netflixtechblog.com
- Uber Engineering Blog — Apache Hudi: https://eng.uber.com/apache-hudi
- HIPAA Data Retention Requirements: https://www.hhs.gov/hipaa
- MLflow — ML Lifecycle Management: https://mlflow.org/docs/latest
- Apache Superset — Open Source BI: https://superset.apache.org

10. Appendix

Detailed Architecture Diagram